Atmosic SDK

Quick Reference Guide

SUMMARY: This document provides overview and instructions on how to build and program applications using the Atmosic Software Development Kit (SDK) with an ATM2/ATM3 or ATM33/e Wireless SoC Series Evaluation Kit (EVK).

CONFIDENTIAL



Atmosic™

CONFIDENTIAL | Doc. No. ATM-QRGSDK-0070

Table of Contents

1. Overview 3

2. SDK and EVK Setup 3

2.1 ATM2/ATM3 EVK 3

2.2 ATM33/e EVK 4

3. ATM2/ATM3 Platform Tools & Utilities 6

3.1 In-System Programming Tool 6

3.2 build_archive 6

3.3 Makefile Helpers 6

3.4 show_archive 8

3.5 burn_archive 8

3.6. ROM and OTP Application 10

3.6.1 ROM and custom application 10

3.6.2 ROM and Flash application 11

3.6.3 Flash operating system 11

4. Make Options and Commands 13

Revision History 15

CONFIDENTIAL

List of Figures

Figure 1 - RDI USB in Device Manager

Figure 2 - Make Executed without Error for BLE_adv Sample Application

Figure 3 - ATM33/e RDI USB in Device Manager

Figure 4 - ATM33 Make Executed without Error for BLE_adv Sample Application

1. Overview

This document provides overview and instructions on how to build and program applications using the Atmosic software development kit (SDK) with an ATM2/ATM3 or ATM33/e evaluation kit (EVK).

For more information about using the EVK, please refer to the

- ATM2/ATM3 Series
  - EVK User Guide for ATMx201/ATMx202
  - EVK User Guide for ATMx221
- ATM33/e Series
  - ATM33/e Series Evaluation Kit User Guide

2. SDK and EVK Setup

2.1 ATM2/ATM3 EVK

1) Install the latest SDK on the host development platform.
   a) Refer to Atmosic SDK Quick Start Guide for SDK installation

2) Check the EVK version is properly identified.
   a) Refer to the latest Atmosic SDK Quick Start Guide for proper SDK platform working directory.

3) Check USB1 port of the interface board is enumerated correctly. See Figure 1.

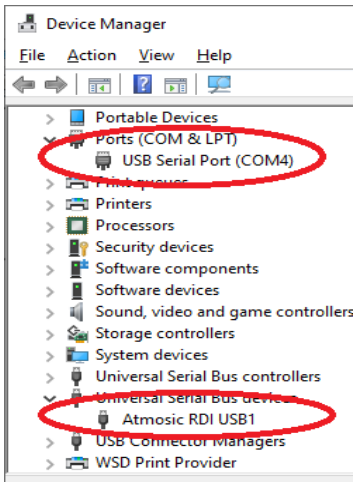


Figure 1 - RDI USB in Device Manager

4) Ensure BLE_adv example can compile without error as shown in Figure 2 after executing the following commands:

CONFIDENTIAL

```
$ cd platform/atm2/ATM22xx-x0x/examples/BLE_adv/
$ make
```

```
/c/AtmosicSDK/x.y.z/tools/gcc-arm-none-eabi-10.3-2021.07/bin/arm-none-eabi-
objcopy -O binary BLE_adv.elf BLE_adv.bin
/c/AtmosicSDK/x.y.z/tools/gcc-arm-none-eabi-10.3-2021.07/bin/arm-none-eabi-
objdump -d BLE_adv.elf >BLE_adv.asm
```

Figure 2 - Make Executed without Error for BLE_adv Sample Application

2.2 ATM33/e EVK

1) SDK is installed on the host development platform.
    a) Refer to SDK Quick Start Guide for SDK installation

2) Check EVK version is correctly identified.
    a) Refer to SDK Quick Start Guide for the correct SDK platform working directory.

3) Connect MicroUSB port to host PC and check BULK interface port of the J-Link OB is enumerated correctly. See Figure 3.

CONFIDENTIAL



Figure 3 - ATM33/e RDI USB in Device Manager

4) Ensure BLE_adv example can compile without error as shown in Figure 4, after executing the following commands:

```
$ cd platform/atm33/ATM33xx-5/examples/BLE_adv/
$ make
```

```
/c/AtmosicSDK/x.y.z/tools/gcc-arm-none-eabi-10.3-2021.07/bin/arm-none-eabi-
objcopy -O binary BLE_adv.elf BLE_adv.bin
/c/AtmosicSDK/x.y.z/tools/gcc-arm-none-eabi-10.3-2021.07/bin/arm-none-eabi-
objdump -d BLE_adv.elf >BLE_adv.asm
```

Figure 4 - ATM33 Make Executed without Error for BLE_adv Sample Application

CONFIDENTIAL

3. ATM2/ATM3 Platform Tools & Utilities

3.1 In-System Programming Tool

The SDK ships with a tool called Atmosic In-System Programming Tool (ISP) for bundling all three types of binaries -- OTP NVDS, flash NVDS, and flash -- into a single binary archive.

The ISP tool, which is also shipped as a stand-alone package, can then be used to unpack the components of the archive and download them on a device.

In every platform example, e.g. BLE_adv_scan, the makefiles have the following targets for building, examining, and programming an ISP archive, respectively.

```
make build_archive  //  requires appropriate nvds file, see below
make show_archive  //  display nvds and image details
make burn_archive  // program files into storage
```

3.2 build_archive

build_archive bundles flash_nvds.bin, otp_nvds.nvm, and $(APP).bin where $(APP) is the name of the application directory.

Not all binaries are mandatory, build_archive can build selective binaries. For example the target $(APP).elf can be chosen instead of the $(APP).bin by setting the make variable `ARCH_FLASH_TYPE=elf`.

***Note**: flash_nvds.bin and $(APP).bin apply only to flash based applications and otp_nvds.nvm applies to OTP storage, see OTP_beacon sample application.*

3.3 Makefile Helpers

Makefile helper allows various application configurations.

CONFIDENTIAL

Key helpers for NVDS:

```
flash_nvds.data := List of NVDS tags required by application
otp_nvds.data: List of NVDS tags for application running in OTP
```

build_archive NVDS requirements:
For flash-based applications that define flash_nvds.data in makefile:

```
make build_flash_nvds  //  creates flash_nvds.bin
```

For OTP based data that define otp_nvds.data in makefile:

```
make pull_otp_nvds  // creates otp_nvds.nvm
make show_pretty_flash_nvds  // shows contents of .nvm file
make rebuild_otp_nvds // creates new otp_nvds.nvm
```

Output of “make build_archive”

```
arch.atm  //  archived application, FW binary plus NVDS (OTP and/or flash)
```

Output of “ make build_archive ARCH_FLASH_TYPE=elf ”

```
arch.atm  // archived application, FW elf plus NVDS (OTP and/or flash)
```

CONFIDENTIAL

3.4 show_archive

Example output of show_archive from WURX_ADV example application below. It shows platform and image size:

```
$ make show_archive
/c/AtmosicSDK/x.y.z/tools/atm_isp decode -i arch.atm
Platform ATMx2xx-0x0 / x1x
LoadFlashNvds image=(size=260,content=b"NVDS\x06\x06'\x00"...)
LoadFlash image=(size=1731732,content=b'\x7fELF\x01\x01\x01\x00'...)
```

3.5 burn_archive

The burn_archive argument programs the archived file into the target storage.The atm_isp tool is included with the SDK and is called when running the “make burn_archive” command. CLI can be used to program an archive file by directly calling the atm_isp tool from the tools directory (see below).

Also, the GUI based atm_isp tool can be used to program archive files without the tools provided in SDK. See “isp_tool_gui” provided on customer portal.

```
make burn_archive  //  programs arch.atm into storage
```

CONFIDENTIAL

Defining the makefile variable BURN_ARCH_VERIFY tells the ISP tool to verify images after loading them on the device.

BURN_ARCH_DEBUG makes the tool more verbose.

If there is OTP data available, setting BURN_ARCH_ERASE_WORKAROUNDS will erase any unlocked workaround tags (0xfc, 0xfd, and 0xfe) from OTP before burning the new contents.

Finally, defining BURN_ARCH_PROGRAM_ONLY prevents reset-hard-on-exit; it can be used to get a behavior similar to "`make program_all`" as opposed to "`make run_all`".

Using atm_isp tool CLI:

```
$cd <SDK>/tools
$ ../atm_isp burn -h
usage: atm_isp burn [-h] [-i ARCHIVE] [-r OPENOCD_PKG_ROOT] [-E] [-e] [-v]
                    [-c] [-t TCL_SCRIPT] [-d DST_DIR] [-p]

optional arguments:
  -h, --help            show this help message and exit
  -i ARCHIVE, --input ARCHIVE
                        Input archive file
  -r OPENOCD_PKG_ROOT, --openocd_pkg_root OPENOCD_PKG_ROOT
                        Path to directory where openocd and its scripts are
                        found
  -E, --openocd_script_only
                        Stop after preparing OpenOCD script
  -e, --erase_workarounds
                        Erase workaround tags in OTP before loading OTP
  -v, --verbose         Verbose mode
  -c, --check_image     Verify OTP/flash image after burning/loading
  -t TCL_SCRIPT, --tcl_script TCL_SCRIPT
                        Path to output Jim Tcl script for use by OpenOCD
                        (generates Jim Tcl script only; delays all
operations
                        post-unpacking of archive to Tcl/OpenOCD); implies
-E
  -d DST_DIR, --dst_dir DST_DIR
                        Use this directory to dump openocd script in;
implies
                        -E
  -p, --program_only    Program the device only (no reset hard on exit)
```

Example:

```
$ ./atm_isp burn -c -v -i <archive.atm>
```

3.6. ROM and OTP Application

The OTP_beacon directory provides an example of this mode of operation. A number of pre-configured beacon examples are provided in:

platform/atm[2,3]/ATM<part>/examples/OTP_beacon/reference_beacons.mk
One can be selected by setting the REF_BCN makefile variable when building nvds.

For example:

```
make REF_BCN:=LR_coded rebuild_otp_nvds -> rebuild OTP based NVDS +
application
make push_otp_nvds -> store into OTP
```

3.6.1 ROM and custom application

The on-board OTP is relatively small (4 KB), but that can be enough for certain custom solutions that want to avoid using FLASH. A special NVDS tag can be programmed into OTP that the ROM will copy into RAM and execute.

The tmp1075_sensor_adv directory shows how small applications can be rapidly developed using FLASH and then deployed to OTP.

Workflow for debug application and NVDS in flash:

```
make run_all
```

Deployment of optimized application and NVDS to OTP:

```
make pull_otp_nvds
make rebuild_otp_nvds
make push_otp_nvds
```

CONFIDENTIAL

3.6.2 ROM and Flash application

In this mode, the ROM and an application on FLASH cooperate to share the chip resources. This allows extremely compact yet full Bluetooth applications to be developed using APIs in ROM.

BLE_adv, BLE_scan, HCI, HID_keyboard, and HID_mouse all use this mode of operation. See Make Options and Commands section for the list of make commands.

3.6.3 Flash operating system

In this mode, the ROM acts as a first stage bootloader for a fully formed operating system on the FLASH. The ROM will configure enough modules to discover the FLASH operating system and map it over the top of the ROM at address 0x00000000.

The image on FLASH begins with the ARM stack pointer, reset vector, and other exception vectors. When the ROM hands execution over to this reset vector, the OS is free to use all chip resources.

By default, the above goals build for and program to the flash on the target. The application will access read-only code and data in place from the flash. For certain applications, this can result in undesirable power consumption and performance.

The AVOID_XIP:=1 option can be specified to locate said code and data in RAM instead, and to have it copied from flash at boot time. Be sure to clean the workspace when switching this option:

```
make clean
make AVOID_XIP:=1 run
```

The makefiles, linker scripts, and openocd scripts default to utilize an entire 4 Mb flash device with the last 32 KB reserved for NVDS.

The FLASH_SIZE and NVDS_SIZE makefile variables can be set on the command line or at the top of the example makefile to accommodate different layouts.

Be sure to clean the workspace when switching either of these options. For example, to build an image suitable for OTP partition programming on a 4 Mb device:

CONFIDENTIAL

```
make clean
make FLASH_SIZE:=0x40000
```

To avoid the use of flash entirely, the RUN_IN_RAM:=1 option can be specified. This is particularly useful for the HCI example in order to perform qualification tests on flashless designs.

As above, be sure to clean the workspace when switching between flash and RAM workflows:

```
make clean
make RUN_IN_RAM:=1 run
```

To run the system entirely out of flash, FLASHROM=ble-full-all option can be specified. When using this option, the ROM acts only as a boot loader.

Advantages when using FlashROM build:

- Most up-to-date Bluetooth LE stack and HW workarounds.
- BLE stack can be optimized for specific application.
- RAM partitioning not needed (Refer to Table 2: Memory Reservations for Third-Party OS Images in SDK User Reference Manual).

Disadvantages when using FlashROM build:

- Increased application binary size.
- Increased power consumption due to more flash accesses.
- Some RAM is used for timing critical code sections.

```
make clean
make FLASHROM:=ble-full-all run
```

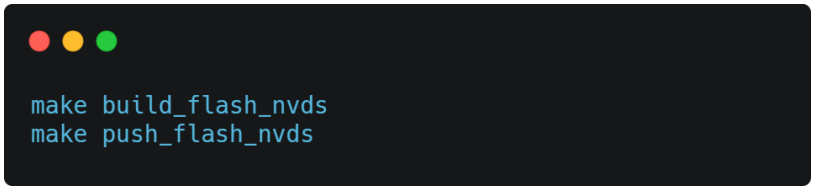
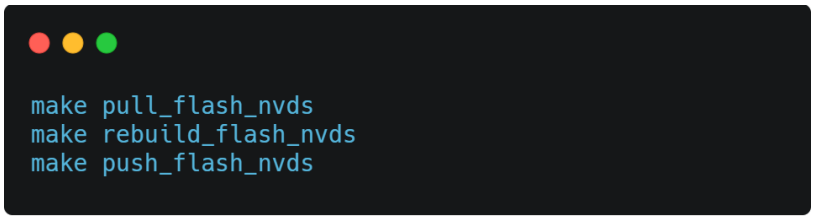
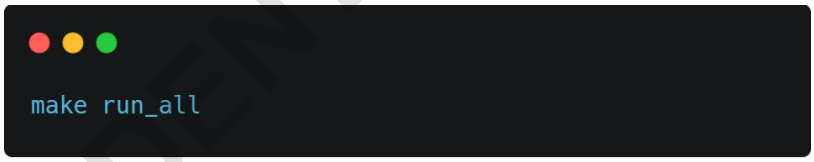
CONFIDENTIAL

4. Make Options and Commands

Make Option	Command
Clean build artifacts	`make clean`
Build flash application	`make`
Build flash application, program hardware, but leave ARM M0/M33 CPU halted	`make program`
Build flash application and compare against flash on target	`make verify`
Inspect OTP NVDS from device	`make pull_otp_nvds` `make show_pretty_otp_nvds`
Inspect flash-based NVDS	`make pull_flash_nvds` `make show_pretty_flash_nvds`

Update OTP NVDS on device	`make pull_otp_nvds`
	`make rebuild_otp_nvds`
	`make push_otp_nvds`

CONFIDENTIAL

Program new flash-based NVDS	
Update flash-based NVDS	
Build flash NVDS, push, build app, and program	
Build flash NVDS, push, build app, and run	
Collect core dump for offline analysis	
Build archive with protected MP region	

CONFIDENTIAL

Revision History

Date	Version	Description
September 15, 2022	0.70	Updated various sections..
May 13, 2022	0.60	Initial version created SDK 5.1.0 release.

CONFIDENTIAL

ATMOSIC TECHNOLOGIES – DISCLAIMER

This product document is intended to be a general informational aid and not a substitute for any literature or labeling accompanying your purchase of the Atmosic product. Atmosic reserves the right to amend its product literature at any time without notice and for any reason, including to improve product design or function. While Atmosic strives to make its documents accurate and current, Atmosic makes no warranty or representation that the information contained in this document is completely accurate, and Atmosic hereby disclaims (i) any and all liability for any errors or inaccuracies contained in any document or in any other product literature and any damages or lost profits resulting therefrom; (ii) any and all liability and responsibility for any action you take or fail to take based on the information contained in this document; and (iii) any and all implied warranties which may attach to this document, including warranties of fitness for particular purpose, non-infringement and merchantability. Consequently, you assume all risk in your use of this document, the Atmosic product, and in any action you take or fail to take based upon the information in this document. Any statements in this document in regard to the suitability of an Atmosic product for certain types of applications are based on Atmosic's general knowledge of typical requirements in generic applications and are not binding statements about the suitability of Atmosic products for any particular application. It is your responsibility as the customer to validate that a particular Atmosic product is suitable for use in a particular application. All content in this document is proprietary, copyrighted, and owned or licensed by Atmosic, and any unauthorized use of content or trademarks contained herein is strictly prohibited.

Copyright ©2022 by Atmosic Technologies. All rights reserved. Atmosic logo is a registered trademark of Atmosic Technologies Inc. All other trademarks are the properties of their respective holders.

CONFIDENTIAL

Atmosic™

Atmosic Technologies | 2105 S. Bascom Ave. | Campbell CA, 95008
www.atmosic.com